

Assignment-11.2

Task 1 – Stack Using AI Guidance

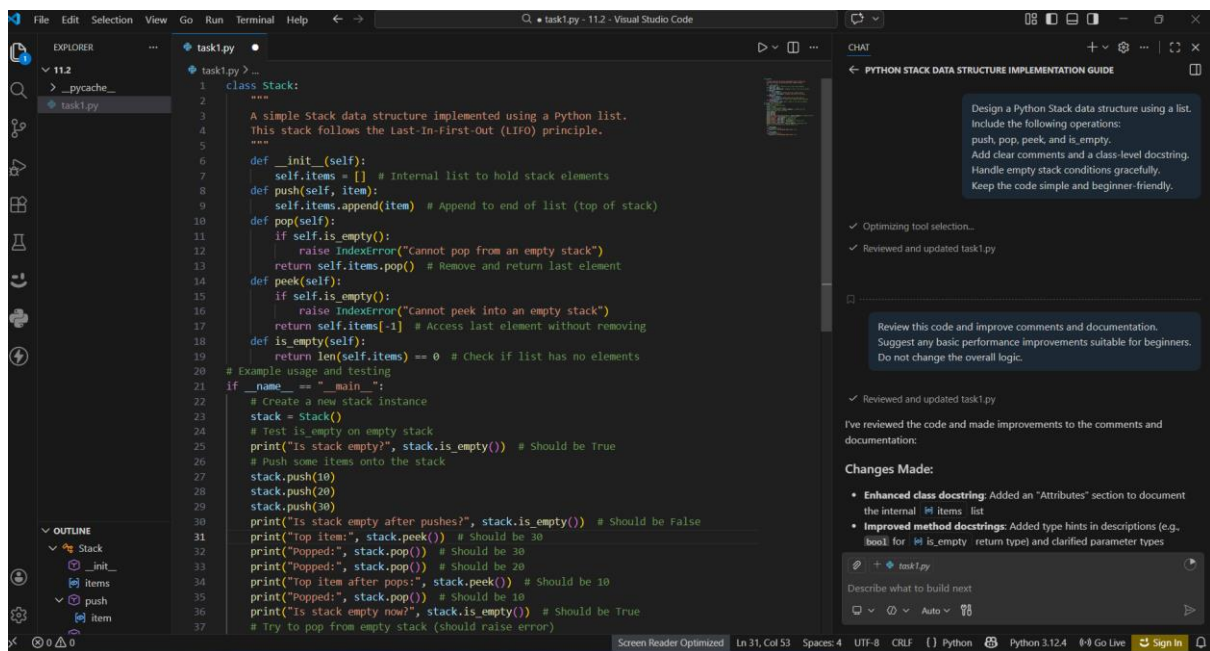
Objective

To design and implement a Stack data structure in Python using AI assistance, supporting basic stack operations such as push, pop, peek, and empty check.

AI Prompt Used

Design a Python Stack data structure using a list.
Include the following operations:
push, pop, peek, and is_empty.
Add clear comments and a class-level docstring.
Handle empty stack conditions gracefully.
Keep the code simple and beginner-friendly.

AI-Generated Code



The screenshot shows a Visual Studio Code editor with a Python file named `task1.py`. The code implements a `Stack` class using a Python list. The class includes methods for `push`, `pop`, `peek`, and `is_empty`. The `push` method appends an item to the list. The `pop` method removes and returns the last element, raising an `IndexError` if the stack is empty. The `peek` method returns the last element without removing it, also raising an `IndexError` if the stack is empty. The `is_empty` method checks if the list is empty. The code includes a main block with example usage and testing.

```
1 class Stack:
2     """
3     A simple Stack data structure implemented using a Python list.
4     This stack follows the Last-In-First-Out (LIFO) principle.
5     """
6     def __init__(self):
7         self.items = [] # Internal list to hold stack elements
8     def push(self, item):
9         self.items.append(item) # Append to end of list (top of stack)
10    def pop(self):
11        if self.is_empty():
12            raise IndexError("Cannot pop from an empty stack")
13        return self.items.pop() # Remove and return last element
14    def peek(self):
15        if self.is_empty():
16            raise IndexError("cannot peek into an empty stack")
17        return self.items[-1] # Access last element without removing
18    def is_empty(self):
19        return len(self.items) == 0 # Check if list has no elements
20    # Example usage and testing
21    if __name__ == "__main__":
22        # Create a new stack instance
23        stack = Stack()
24        # Test is_empty on empty stack
25        print("Is stack empty?", stack.is_empty()) # Should be True
26        # Push some items onto the stack
27        stack.push(10)
28        stack.push(20)
29        stack.push(30)
30        print("Is stack empty after pushes?", stack.is_empty()) # Should be False
31        print("Top item:", stack.peek()) # Should be 30
32        print("Popped:", stack.pop()) # Should be 30
33        print("Popped:", stack.pop()) # Should be 20
34        print("Top item after pops:", stack.peek()) # Should be 10
35        print("Popped:", stack.pop()) # Should be 10
36        print("Is stack empty now?", stack.is_empty()) # Should be True
37        # Try to pop from empty stack (should raise error)
```

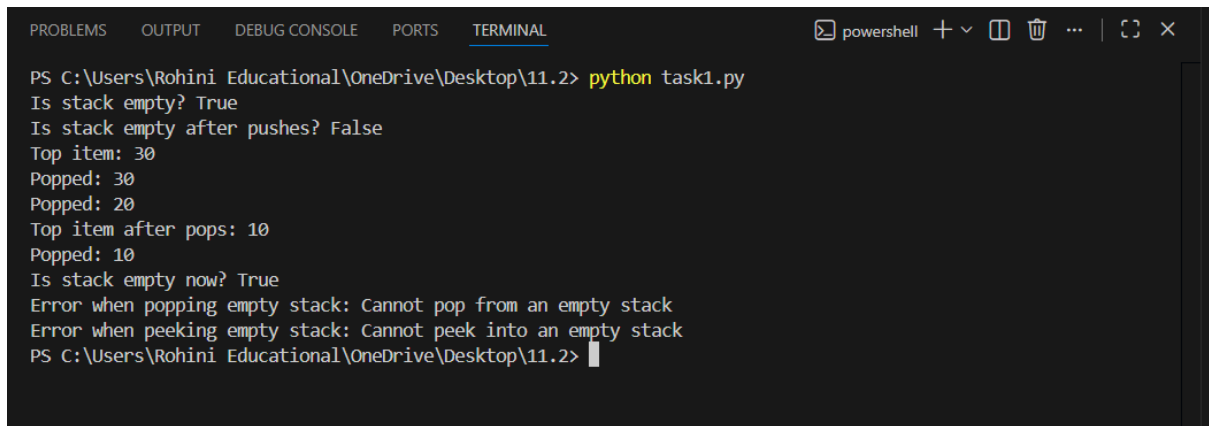
The AI chat window on the right shows the prompt and the generated code. It also includes a "Changes Made" section with the following updates:

- Enhanced class docstring: Added an "Attributes" section to document the internal `items` list
- Improved method docstrings: Added type hints in descriptions (e.g., `bool` for `is_empty` return type) and clarified parameter types

Explanation

- Stack follows the Last In First Out (LIFO) principle.
- Python list is used to store stack elements.
- `push()` adds an element to the stack.
- `pop()` removes the top element and raises an error if the stack is empty.
- `peek()` returns the top element without removing it.

OUTPUT

A screenshot of a Visual Studio Code terminal window. The terminal title bar shows 'powershell' and standard window controls. The terminal content shows a PowerShell prompt at 'C:\Users\Rohini Educational\OneDrive\Desktop\11.2>' where the command 'python task1.py' has been executed. The output of the script is as follows:

```
PS C:\Users\Rohini Educational\OneDrive\Desktop\11.2> python task1.py
Is stack empty? True
Is stack empty after pushes? False
Top item: 30
Popped: 30
Popped: 20
Top item after pops: 10
Popped: 10
Is stack empty now? True
Error when popping empty stack: Cannot pop from an empty stack
Error when peeking empty stack: Cannot peek into an empty stack
PS C:\Users\Rohini Educational\OneDrive\Desktop\11.2>
```

Result

The Stack data structure was successfully implemented and executed in VS Code using AI guidance.

Task 2 – Queue Design Using AI Guidance

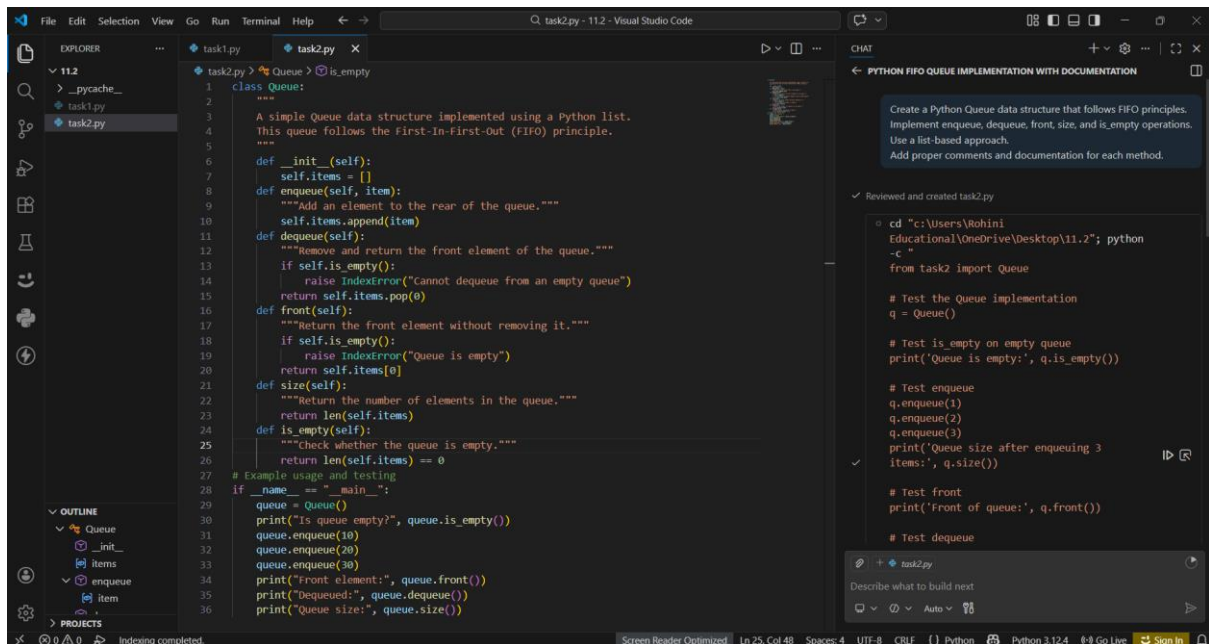
Objective

To design and implement a Queue data structure in Python using AI assistance, following the First-In-First-Out (FIFO) principle and supporting basic queue operations.

AI Prompt Used

Create a Python Queue data structure that follows FIFO principles.
Implement enqueue, dequeue, front, size, and is_empty operations.
Use a list-based approach.
Add proper comments and documentation for each method.

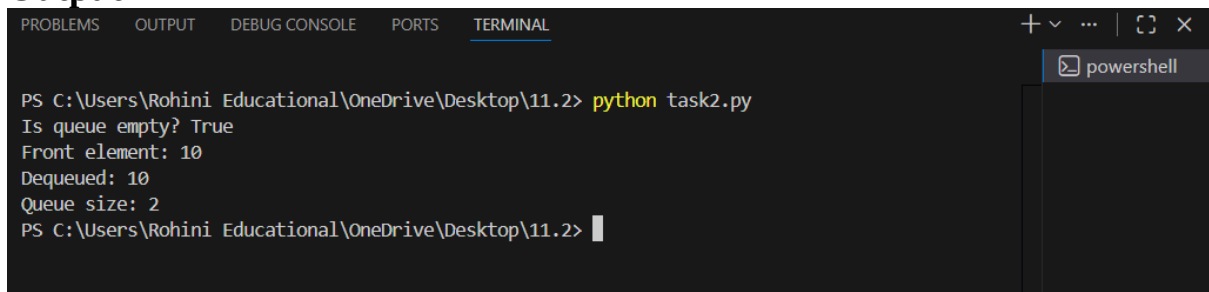
AI-Generated Code



Explanation

- Queue follows the FIFO (First In First Out) principle.
- Python list is used to store queue elements.
- enqueue() adds elements to the rear of the queue.
- dequeue() removes the front element and raises an error if the queue is empty.
- front() returns the first element without removing it.

Output



Result

The Queue data structure was successfully implemented and executed in VS Code using AI guidance.

Task 3 – Singly Linked List Construction Using AI Guidance

Objective

To design and implement a Singly Linked List in Python with AI assistance, supporting node creation, insertion, and traversal.

AI Prompt Used

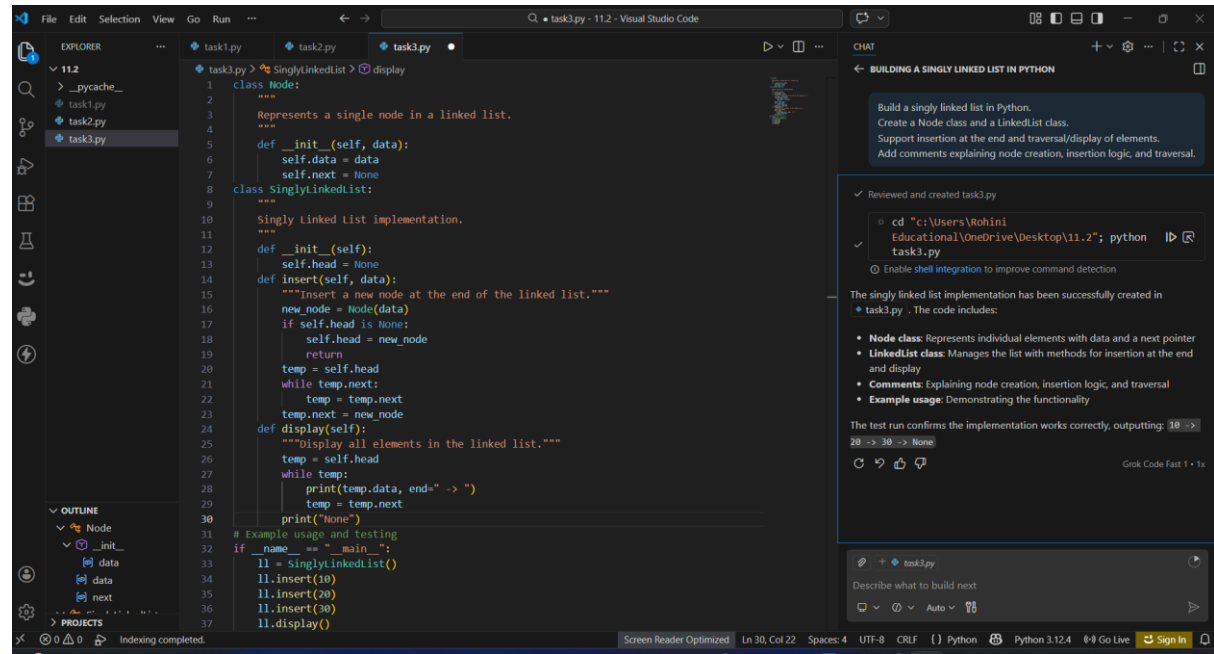
Build a singly linked list in Python.

Create a Node class and a LinkedList class.

Support insertion at the end and traversal/display of elements.

Add comments explaining node creation, insertion logic, and traversal.

AI-Generated Code

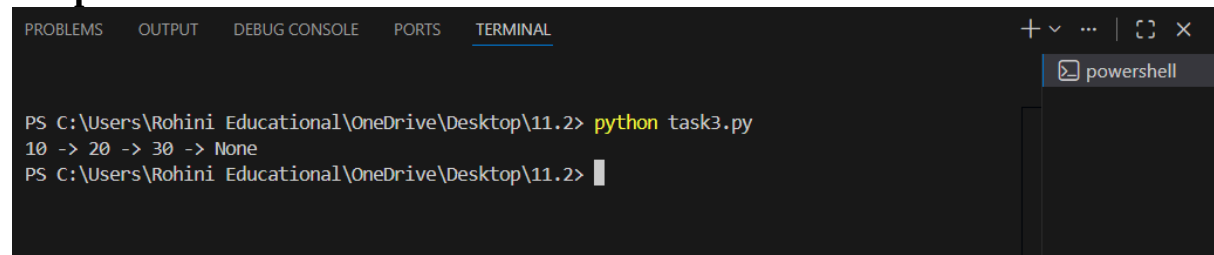


```
1 class Node:
2     """
3     Represents a single node in a linked list.
4     """
5     def __init__(self, data):
6         self.data = data
7         self.next = None
8 class SinglyLinkedList:
9     """
10    Singly Linked List implementation.
11    """
12    def __init__(self):
13        self.head = None
14    def insert(self, data):
15        """Insert a new node at the end of the linked list."""
16        new_node = Node(data)
17        if self.head is None:
18            self.head = new_node
19            return
20        temp = self.head
21        while temp.next:
22            temp = temp.next
23        temp.next = new_node
24    def display(self):
25        """Display all elements in the linked list."""
26        temp = self.head
27        while temp:
28            print(temp.data, end=" -> ")
29            temp = temp.next
30        print("None")
31
32 # Example usage and testing
33 if __name__ == "__main__":
34     ll = SinglyLinkedList()
35     ll.insert(10)
36     ll.insert(20)
37     ll.insert(30)
38     ll.display()
```

Explanation

- A linked list is made up of nodes connected by pointers.
- Each node contains data and a reference to the next node.
- insert() adds a new node at the end of the list.
- display() traverses the list and prints all elements.
- This implementation uses dynamic memory allocation.

Output



```
PS C:\Users\Rohini Educational\OneDrive\Desktop\11.2> python task3.py
10 -> 20 -> 30 -> None
PS C:\Users\Rohini Educational\OneDrive\Desktop\11.2>
```

Result

The Singly Linked List was successfully implemented and executed in VS Code using AI guidance.

Task 4 – Binary Search Tree Operations Using AI Guidance

Objective

To design and implement a Binary Search Tree (BST) in Python using AI assistance, focusing on node insertion and in-order traversal.

AI Prompt Used

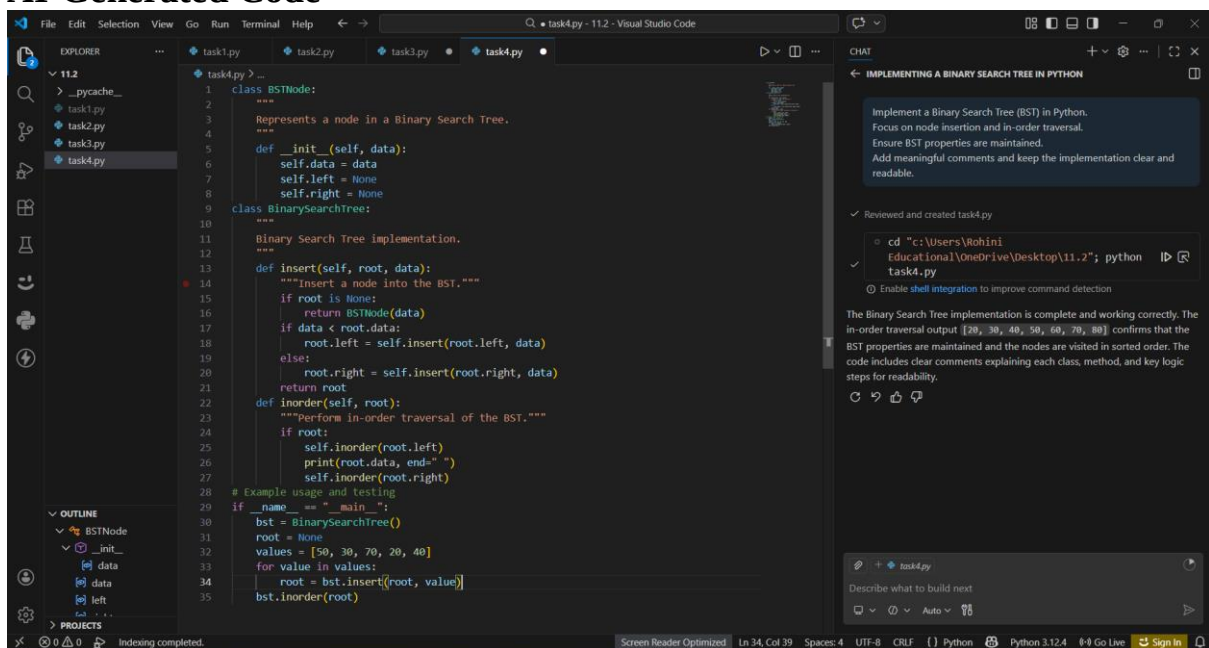
Implement a Binary Search Tree (BST) in Python.

Focus on node insertion and in-order traversal.

Ensure BST properties are maintained.

Add meaningful comments and keep the implementation clear and readable.

AI-Generated Code

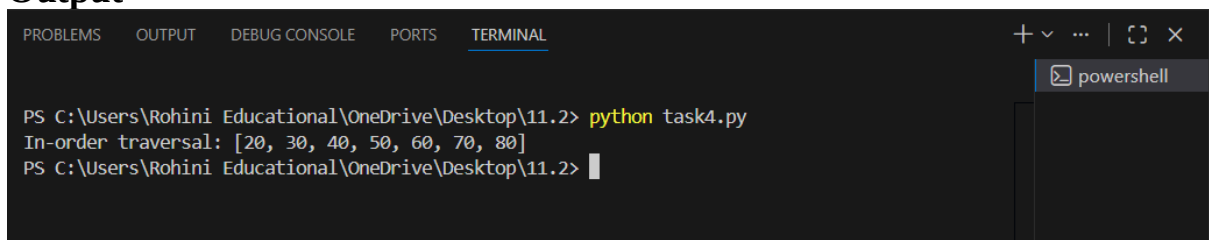


```
1 class BSTNode:
2     """
3     Represents a node in a Binary Search Tree.
4     """
5     def __init__(self, data):
6         self.data = data
7         self.left = None
8         self.right = None
9
10 class BinarySearchTree:
11     """
12     Binary Search Tree implementation.
13     """
14     def insert(self, root, data):
15         """Insert a node into the BST."""
16         if root is None:
17             return BSTNode(data)
18         if data < root.data:
19             root.left = self.insert(root.left, data)
20         else:
21             root.right = self.insert(root.right, data)
22         return root
23
24     def inorder(self, root):
25         """Perform in-order traversal of the BST."""
26         if root:
27             self.inorder(root.left)
28             print(root.data, end=" ")
29             self.inorder(root.right)
30
31 # Example usage and testing
32 if __name__ == "__main__":
33     bst = BinarySearchTree()
34     root = None
35     values = [50, 30, 70, 20, 40]
36     for value in values:
37         root = bst.insert(root, value)
38     bst.inorder(root)
```

Explanation

- A binary search tree maintains sorted data.
- Left subtree contains values smaller than the root.
- Right subtree contains values greater than the root.
- insert() places nodes according to BST rules.
- In-order traversal prints elements in ascending order.

Output



```
PS C:\Users\Rohini Educational\OneDrive\Desktop\11.2> python task4.py
In-order traversal: [20, 30, 40, 50, 60, 70, 80]
PS C:\Users\Rohini Educational\OneDrive\Desktop\11.2>
```

Result

The Binary Search Tree was successfully implemented and executed in VS Code using AI guidance.

Task 5 – Hash Table Implementation Using AI Guidance

Objective

To design and implement a Hash Table in Python with AI assistance, supporting insert, search, and delete operations, with collision handling via chaining.

AI Prompt Used

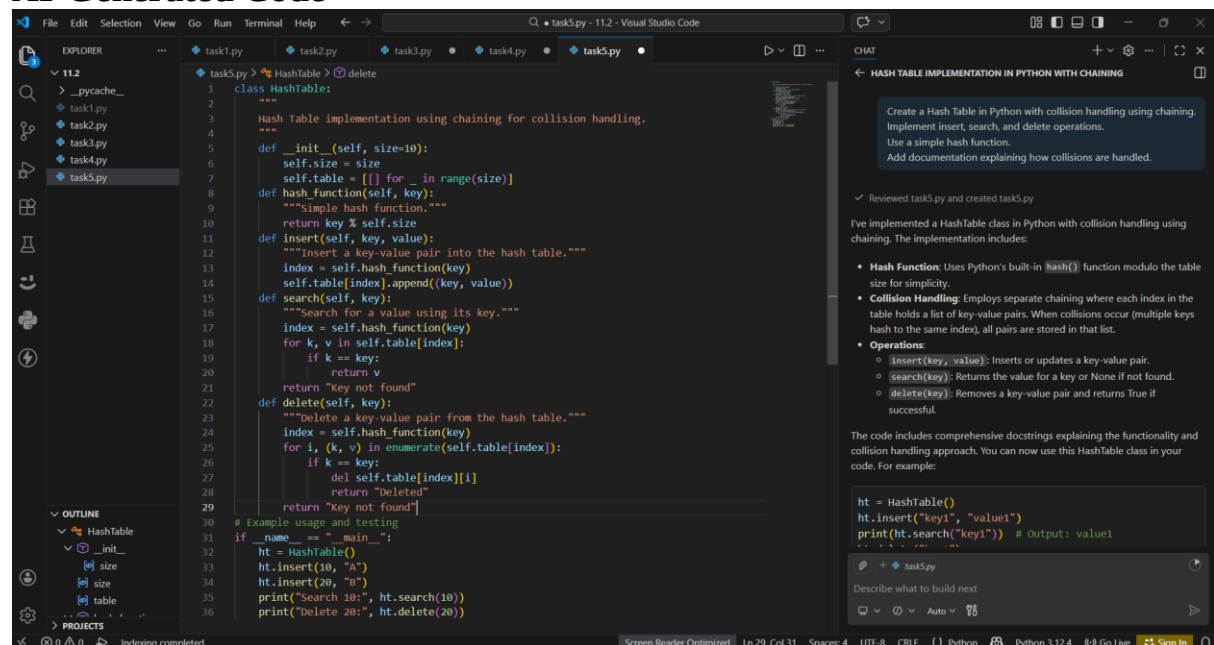
Create a Hash Table in Python with collision handling using chaining.

Implement insert, search, and delete operations.

Use a simple hash function.

Add documentation explaining how collisions are handled.

AI-Generated Code

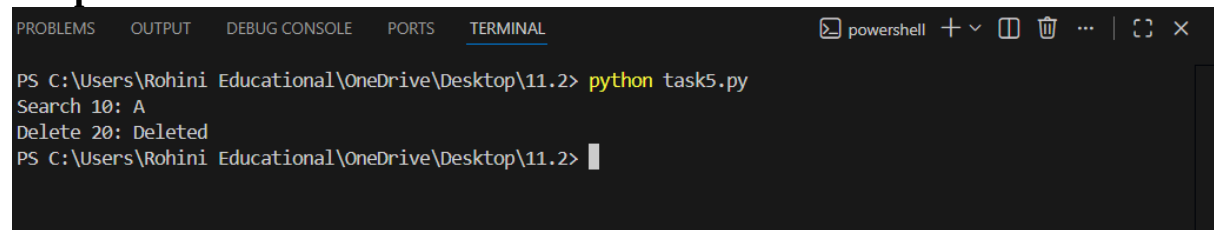


```
class HashTable:
    """
    Hash Table Implementation using chaining for collision handling.
    """
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]
    def hash_function(self, key):
        """Simple hash function."""
        return key % self.size
    def insert(self, key, value):
        """Insert a key-value pair into the hash table."""
        index = self.hash_function(key)
        self.table[index].append((key, value))
    def search(self, key):
        """Search for a value using its key."""
        index = self.hash_function(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return "Key not found"
    def delete(self, key):
        """Delete a key-value pair from the hash table."""
        index = self.hash_function(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]
                return "Deleted"
        return "Key not found"
# Example usage and testing
if __name__ == "__main__":
    ht = HashTable()
    ht.insert(10, "A")
    ht.insert(20, "B")
    print("Search 10:", ht.search(10))
    print("Delete 20:", ht.delete(20))
```

Explanation

- Hash Table stores data in key-value pairs.
- A hash function maps keys to table indices.
- Chaining is used to handle collisions by storing multiple elements at the same index.
- insert() adds data to the table.
- search() retrieves values using keys.
- delete() removes key-value pairs safely.

Output



```
PS C:\Users\Rohini Educational\OneDrive\Desktop\11.2> python task5.py
Search 10: A
Delete 20: Deleted
PS C:\Users\Rohini Educational\OneDrive\Desktop\11.2>
```

Result

The Hash Table was successfully implemented and executed in VS Code using AI guidance.